ENPH 353 Final Report
Team #20 “TT77”

Team Members:

Matan Guttmann, 35245166

Simon Ryu, 93031862

A little note about our team name “TT77”:

The day of the competition we kept occasionally mislabelling license plates in the inner ring. Whenever we mislabelled a plate, the predicted label was “TT77”. After some investigation, we realised that the perspective transform was failing. Making a change to a threshold in the perspective transform script, we were able to resolve the problem, but “TT77” remained our deepest fear.

1 Overview

The goal of the final project in ENPH353 was to build an autonomous agent that could navigate a course, identify characters on license plates of parked cars and avoid NPC obstacles. There are many different ways to approach this problem. We decided to follow the simplest, but most robust approach we could think of.

The license plate character identification is performed by first performing a perspective transform, using classical techniques, and then using a Convolutional Neural Network (CNN) to identify the individual characters which make up a plate.

The control of the agent/robot is achieved using a mixture of PID and hard coding, for different sections for the course.

2 License Plate Character Identification

There are three distinct steps to how we approached developing our license plate character identification algorithm. First, we developed a workflow, using classical techniques, to apply a perspective transform to images of backs of cars, and to segment parking IDs and license plates into individual characters. We then developed training and validation datasets for our model training. Next, we implemented and optimized a CNN training script to obtain a model which performed adequately for our validation data. Finally, we integrated the classical techniques as well as the model obtained from the CNN training into our controller for the agent.

2.1 Perspective Transform

We used a SIFT homography algorithm in lab 4 of this course, so we decided to start by trying to use this algorithm to obtain a perspective transform matrix.

We first tried using a generated image of the back of a car, with no parking ID or license plate, as our template image. For our test image we used images of backs of cars from the simulation. There were no common keypoints identified between the two classes of images so we could not perform a homography, as seen in fig 1. Particularly, we noticed that SIFT did not identify the corners of the rectangles in the template image, which seem like obvious keypoints. Our brief conclusion from this was that it is likely that SIFT only works well for images that have many local keypoints and our simple generated image did not fit this criteria.

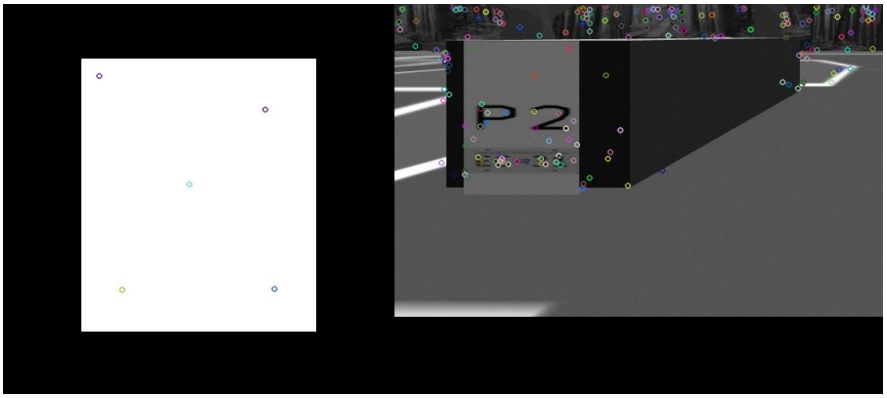


Fig 1. Keypoints of simple generated template (left) and test image from the simulation (right)

We then tested out the SIFT algorithm using a head-on picture of the back of the car in the simulation as our template image. The result of this was that we could perform a good homography on the test image when the parking ID was the same as the template image, but the license plate was different. One option would have been to set up different template images for each parking ID 1-8. The downside of this is that we would need to

know our location on the course. If we missed a car by accident, and were subsequently using the incorrect template image, we would be unable to perform successful perspective transforms for the remainder of the course. We therefore concluded that we would not pursue this idea.

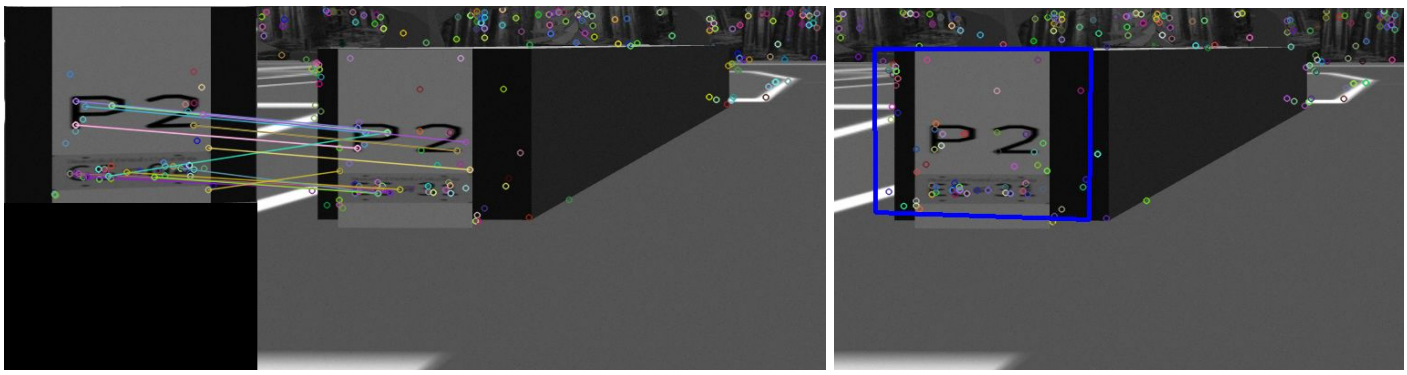
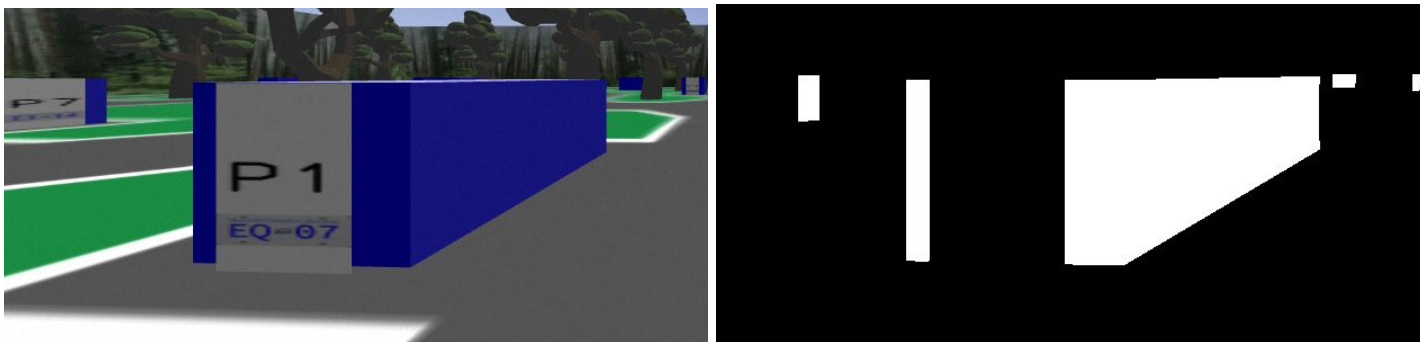


Fig 2. (Left) Common keypoints of head-on simulation template and off-angle training image from the simulation. (Right) Resulting homography.

We instead decided to try a simpler approach to isolate and “flatten” the region of the image with the parking ID and license plate. The method we ended up using is as follows:

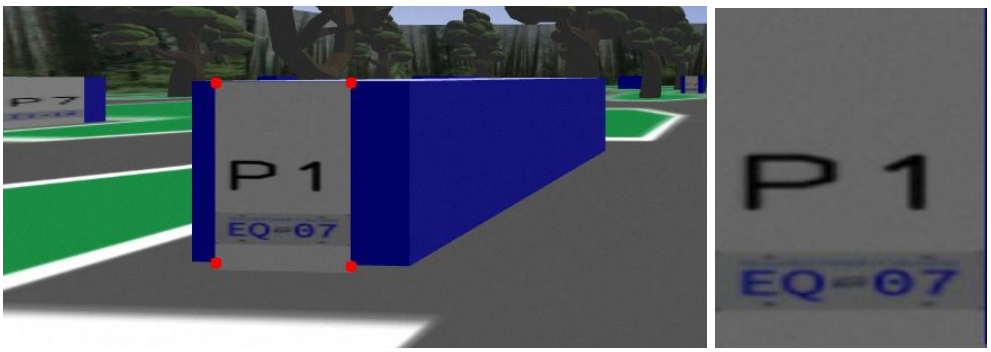
1. Apply an HSV mask on the image, to isolate the pixels of the image which match blue cars. Clean up this mask by applying erosion and dilation.
2. Determine the 4 corners of the white backing for the parking ID and license plate
    a. Take a slice of the image from the lowest point in the mask to 40 pixels above the lowest point. Take the sum of the pixel values in the slice along **axis=0** and apply a threshold. From this we extract the location of the 4 vertical edges in the image by looking for low-to-high and high-to-low transitions.
    b. The two vertical edges which surround the white backing for the parking ID and license plate will be the 2nd and 3rd edges. We call these vertical edges **inner_vert_edges**. These are the x-coordinates of the corners of the white backing.
    c. We then need to determine the y-coordinates of the 4 corners. To do this we take a thin slice of the blue backing of the car to the outside of each inner edge. Taking the sum of pixel values along **axis=1**, thresholding and then looking for low-to-high and high-to-low transitions, we determine the y-coordinate of each of the 4 corners
3. We then pass these 4 corners into a 4 point transform function we found online. This function determines the necessary perspective transform matrix, and applies it to the input image using **cv2.warpPerspective**, in order to “flatten” the off angle image of the license plate and parking ID. The output images were resized to always have the same (200, 160) size. https://www.pyimagesearch.com/2014/08/25/4-point-opencv-getperspective-transf # orm-example/

In the cnn_training repository, this is implemented as a python script called **perspective_transform_script.py**, for running perspective transforms on individual images. This same script is copied to the robot_controllers repository, with some small modifications.



1. (Left) Original Image of Back of Car (Right) Image after HSV mask and erosion and dilation

2. a,b Using bottom 40 pixels of mask to determine x-coordinates of the corners of the white backing



(Left) 2. c. Location of each of the 4 bounding points (Right) 3. Output of 4-point transform.

Fig 3. Perspective transform steps

2.2 Character Segmentation

After we obtained “flattened” images of parking IDs and license plates, we proceeded to segment these images into individual character images. We also converted these individual images to binary, to reduce the required complexity of the CNN. We found that setting the binary threshold to 0.83*(average pixel value) yielded good binary images of characters for all locations on the course.

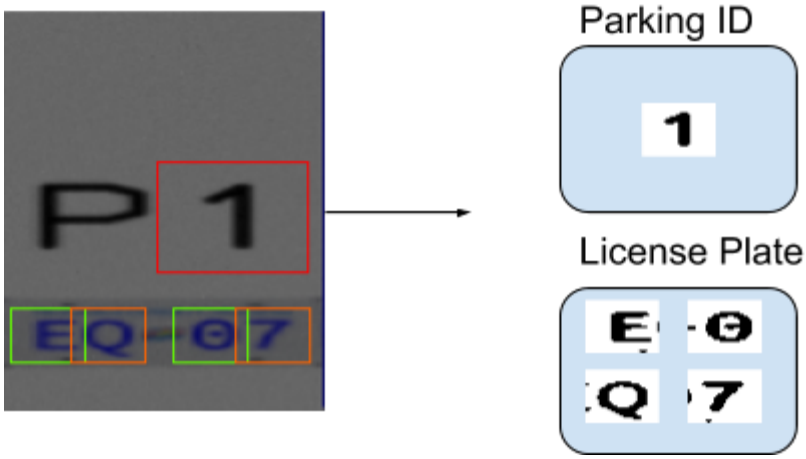


Fig 4. Character segmentation

2.3 Synthetic Data Generation - Training Dataset

To reduce the amount manual collection necessary, we decided to try using synthetic data. Our synthetic data generator is based on the plate generator script Miti provided

2020T1_competition/enph353/enph353_gazebo/scripts/plate_generator.py , with some modifications. The script **cnn_training/plate_generator.py** takes in one integer as an input argument which specifies the amount of plates to randomly generate. The script creates the specified number of plates and directly segments them into characters.

We then augmented this data as part of our CNN training script **cnn_training/plate_detector_cnn_training.py**. We apply augmentation techniques which are a part of the keras datagen package, as well as a custom preprocessing function. This function reduces the resolution of the image, then increases it again. The goal here is to introduce similar artifacts to the characters in the simulation (where we zoom in on low resolution characters). We also apply a gaussian blur and then threshold to restore the image to binary.

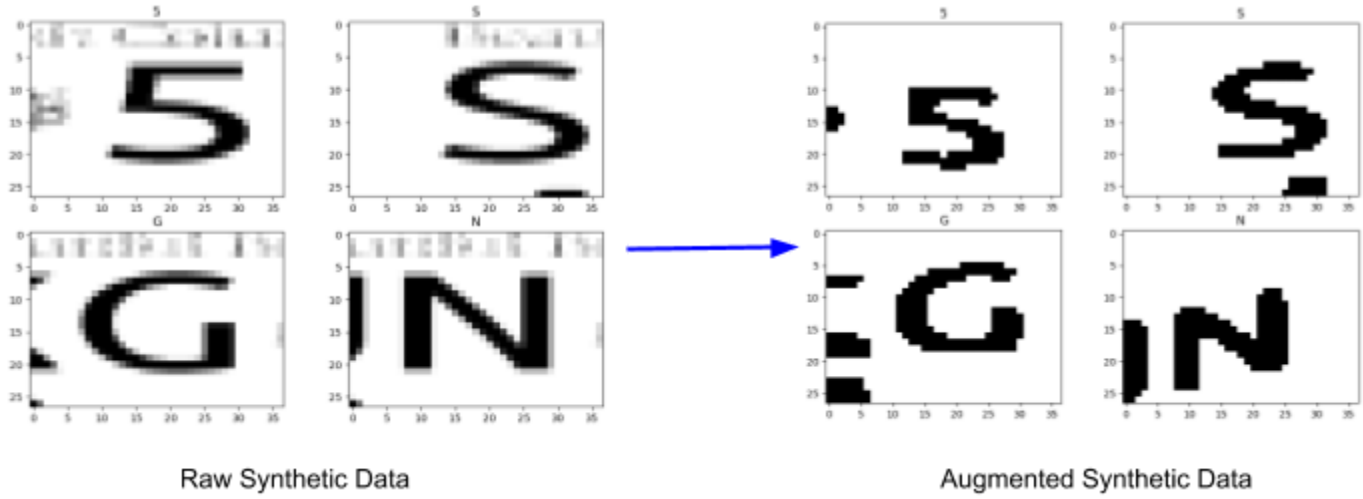


Fig 5. Data augmentation

2.4 Validation Dataset

We collected our validation dataset by taking images of the back of the cars in the simulation. To accelerate this process, we ran the **robot_controller.py** script to control the agent, and used a script **save_rqt_image.py** to save the latest image published to "/R1/pi_camera/image_raw" topic and labelled it with a command line argument. We looked at the csv generated at **2020T1_competition/enph353/enph353_gazebo/scripts** and the location of the robot on the course, to tag the data effectively. Due to the timeline of our data collection, our validation dataset is only made up of images from the outer ring of the course. After we obtained a sufficient number of images of the backs of cars from the simulation, we ran these images through our perspective transform and character segmentation steps, described above.

2.5 CNN training

We trained our character recognition CNN using the **cnn_training/training_scripts/plate_detector_cnn_training.py** script and the training and validation datasets described above. To analyze the performance of our model, we looked at our training and validation loss and accuracy as well as a “modified” confusion matrix and accuracy metric (described in **2.6**). For our “test” dataset, we implemented our model in the robot controller (described in section **3**).

It took a significant amount of tuning to our training script and our datasets to get the model to a point where it achieved sufficient accuracy. Below we present the performance of our model at a few different key points in our development process. Our final CNN architecture (as well as parameters affecting input data) is

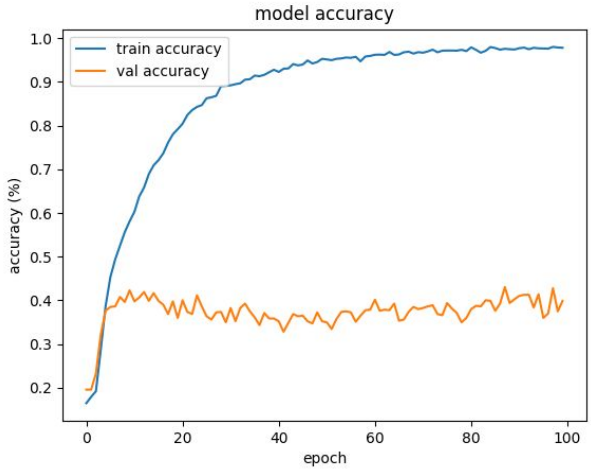
as follows. Highlighted, are parameters that we found particularly important in achieving the performance of our model.

			Values we did not optimize	Value in Final Model
Datasets	Training	size		1000 synthetic plates (then culled)
		composition		Only plate characters, culled and expanded based on worst performance.
	Validation	composition		
		size		240 plates from simulation (then culled)
		binary threshold		0.83*(average pixel value)
Data Augmentation		shear_range		14
		preprocessing function		included
Model	model layer 1	conv2d	8, (5,5)	included
	model layer 2	max_pooling2D	(2,2)	not included
	model layer 3	conv2d	16, (5,5)	included
	model layer 4	max_pooling2D	(2,2)	included
	model layer 5	flatten		included
	model layer 6	dense	500	included
	model layer 7	dense	300	included
	model layer 8	dense	36	included

Fig 6. CNN Architecture

In our initial attempt, we trained the model with the parking ID and plate characters from 500 generated plates and validated the model with parking ID and plate characters from 120 plates from the simulation. Our binary threshold was 0.9*(average pixel value), which led to more saturated and less precise characters as seen in Fig 8. We did not have the preprocessing function for data augmentation described in **2.4**. Finally, we

included model layer 2.



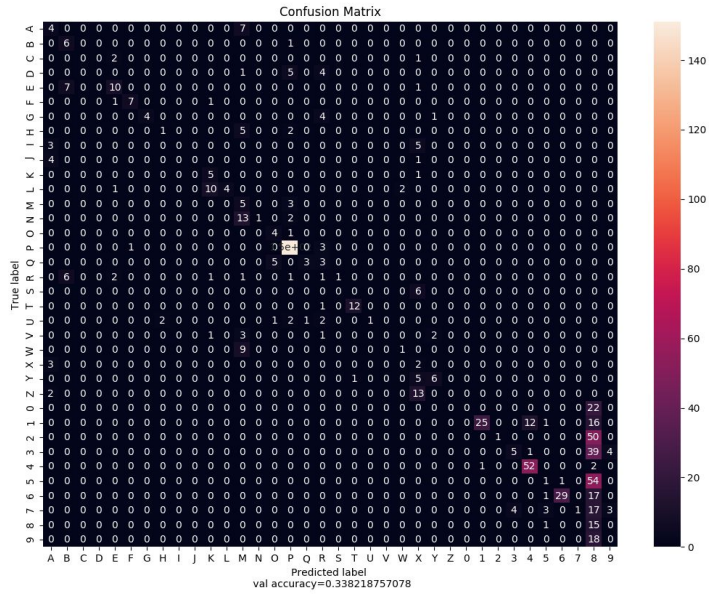
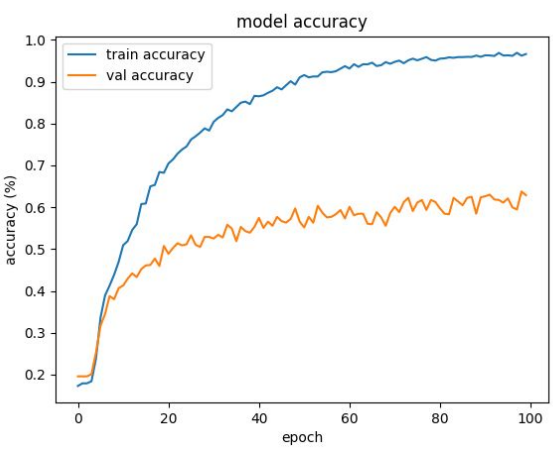


Fig 7a. First Attempt for CNN training. Modified Validation Accuracy = 34%

We then tried increasing the size of our training dataset to 1000 generated plates, and only trained off of plate characters. We increased the size of our validation dataset to 240 plates from the simulation, and also only retained plate characters. We also reduced our binary threshold to 0.83*(average pixel value), which led to less saturated and more precise characters, as seen in fig 8. Finally, we also included a preprocessing function for data augmentation, but with no randomness.



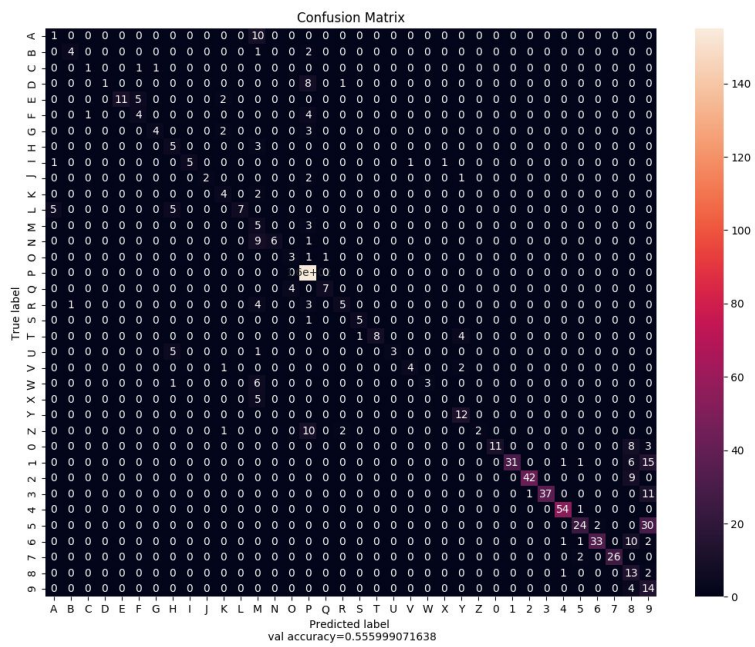


Fig 7b. Second Attempt for CNN training. Modified Validation Accuracy = 56%

Finally, we tried “culling” our dataset, so it included equal amounts of every character. We looked at the results from this, and then included more training and validation examples where the model performed poorly based on the confusion matrix. We added some more randomness to the preprocessing function as well as well as increased the shear_range parameter to 14. Finally, we removed model layer 2.

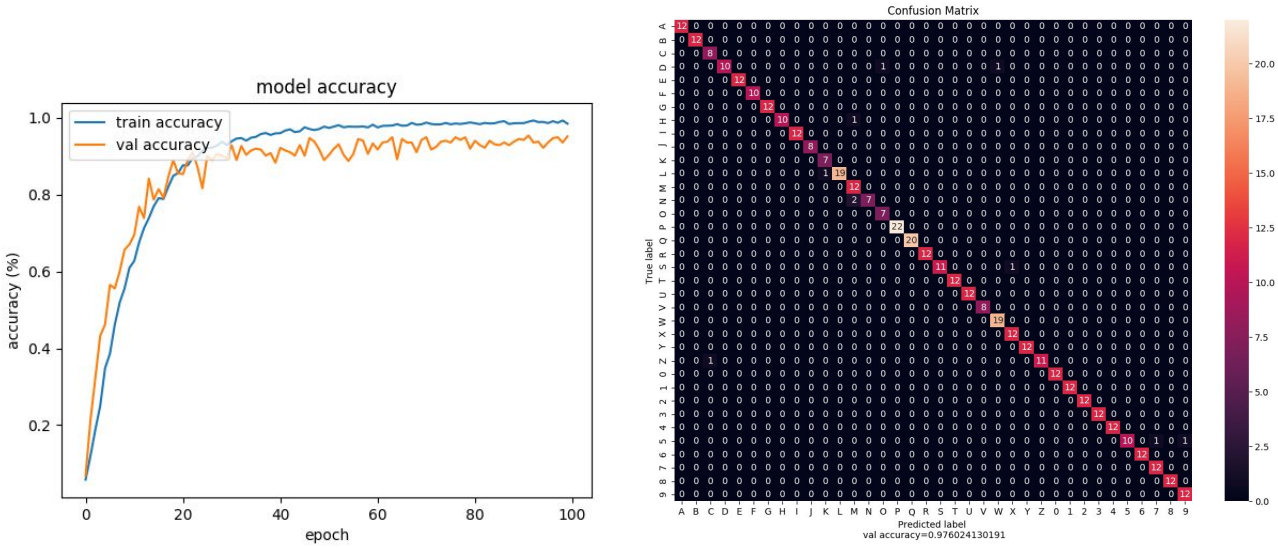


Fig 7c. Final Attempt for CNN training. Modified Validation Accuracy = 97.6%.

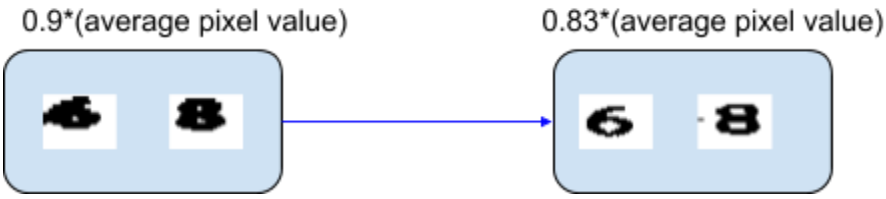


Fig 8. Modification of binary threshold in character segmentation step

2.6 Model Interpretation

We use a single model, to predict labels for both numeric and alphabetic characters. However, we are able to distinguish whether a certain input is alphabetic or numeric based on the characters position in the license plate or if it's a character for the parking ID. Therefore, when we go from categorical probabilities (output of **model.predict(image)**) to a categorical prediction, we only consider probabilities for the associated input “class” (numeric or alphabetic). This is why you do not see any numeric characters being labelled as alphabetic characters, and vice versa, in the confusion matrix. The same method is used in the robot controller.

The validation accuracy computed in the confusion matrix is the (# of good predictions)/(total # of predictions) when the model is used to predict labels for the validation dataset. This was typically about 3-5% higher than the validation accuracy computed by keras.

3 Robot Control Method

The agent uses classical computer vision techniques with PID control to navigate through the course. We designed a finite-state machine to perform a predetermined sequence of actions such as pedestrian/vehicle detection and plate detection. The agent subscribes to Camera and Clock topics, and publishes to velocity commands as well as license plate readings. It is implemented in **robot_controller.py**.

3.1 Driving Control

Initially, the robot uses a hard coded rotation in order to align itself with the outermost line of the outer ring. From then on, it uses a line following algorithm similar to the one delivered in Lab03 with some modifications:

1. When driving in the outside ring, we only consider the right-side of image input to find the center of the line and vice versa for the inside ring.
2. Since the robot needs to stay on track and follow along the line instead of being centered on it, the target point is set at 480px to the left of the middle of the line.

When turning into the inside ring, we use the same method but now set to stay 480px to the right of the inner line. After successfully merging into the inside ring, the robot uses PID control off the innermost line to navigate through the inside ring.

3.2 Pedestrian and Vehicle Detection

During the outside ring state, it constantly checks a single pixel value to detect the color red (B,G < 100, R > 200) located in the bottom of the image. After a number of simulations, I decided that it would be unnecessary to develop more sophisticated techniques such as detecting for color change in a 2D image array, for example.

Similarly, after stopping, we check a pixel value (x=650, y=430) to detect for a pedestrian after binary thresholding the camera image. The robot then drives straight until a red line has been detected again, and we transition back to PID control state.

Fig 8. Thresholded image during pedestrian detection; the center of gray circle is 255 when pedestrian is not on it due to the crosswalk marking, and changes to 0 when the pedestrian is detected.

For vehicle detection, checking a single pixel value was determined to be inaccurate due to shadows and its shaky movements. We ended up finding the average value of the thresholded region of the image to detect a color change. After the vehicle has been detected, the robot waits for additional 3 seconds until the vehicle has completely passed, and then it starts driving.

3.3 Parked Car detection/Plate readings

To detect the parked vehicles, we now take a 2D array (`cv_image[375:400, 400:500]` for outside ring, `cv_image[380:400, 780:840]` for inside ring) and find the average color of the matrix. Their locations were determined in the simulation by finding the locations that provided clear and undistorted views of the license plates. If the average color is within the range of blue, we can perform perspective transform of the images.

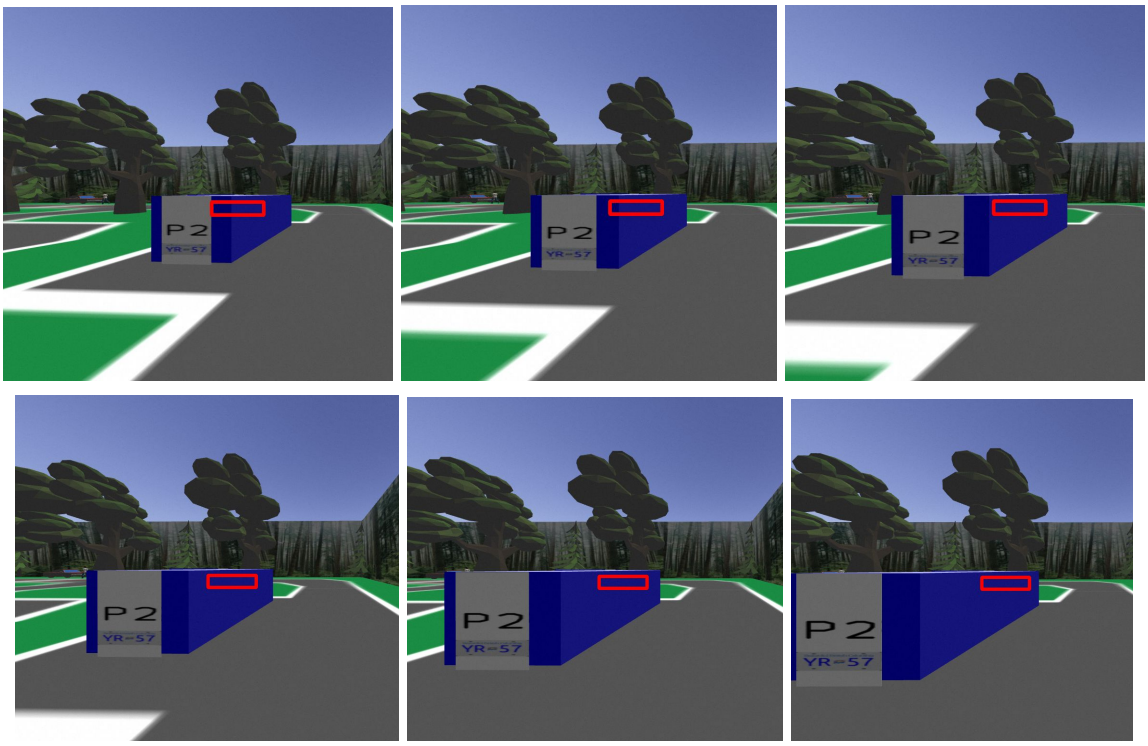


Fig 9. Series of images where the average color of the defined region `([375:400, 400:500]`, shown in red rectangle) is blue.

Using this method, the agent is able to obtain multiple license plate readings for a single parking location. We experimented a couple different techniques on deciding which reading to be submitted, such as publishing the unaltered readings (including the parking location) and publishing if the same id has been read consecutively for more than 3 times. Here is the method we ended up using:

1. Since the reading of parking IDs is more reliable than the license plate readings (1-8 vs. 0-9+A-Z), everytime a license plate has been read, store its ID to the corresponding parking location list.
2. Repeatedly publish the most common ID of the parking location list.

Using this method takes care of incorrect ID readings as well as incorrect location readings, as correct readings will eventually outnumber them.

3.3 Possible Improvements

a. It would have been more reliable to narrow down the labels to (1-6) for outside and (7,8) for inside rings. However, parking location reading seemed consistent enough that I did not end up adopting this.
b. Speed could have been increased. This would require early detections of red lines and corners (where we need to slow down) as significant deceleration can cause the robot to flip over.

Links to Code Repositories

Our repository structure was two separate repositories, one for cnn training related tasks and one for the robot controller. There is some overlap between these two repositories, as the perspective transform and character segmentation was both used for developing the validation dataset, and in “real-time” for the robot controller

cnn_training: https://github.com/thelongcommute/cnn_training
robot_controllers: https://github.com/thelongcommute/robot_controllers

The training data can be obtained downloading and unzipping the file from the following link:
https://drive.google.com/file/d/1uggD_tpvas8vq9OJPlnWmAMk11Id_mSV/view?usp=sharing